

CSE 144 Final Project Report

Transfer Learning Challenge

Ayush Manocha, amanocha@ucsc.edu, 1957614

Spring 2026

1 Introduction

This project involves a 100-class image classification problem on a small, sampled dataset. There are a total of 1,079 training images, with around 10 images per class, and 1,036 unlabeled test images. The goal is to predict the correct class for each test image.

Because this is quite a small dataset, with only 10 examples per class, training a deep neural network from scratch would not be viable. The model would not be able to learn meaningful features well enough from such little data, and it would be highly inaccurate in its predictions. Instead, we use transfer learning to form a more accurate predictive model. With transfer learning, we start from a network that has already been pretrained on millions of images. Due to this extensive training, the model has already learned meaningful general-purpose visual features. We must simply fine-tune this existing network in order to learn higher-order features that are more specific to the 100 classes in our dataset. Given that we are now simply fine-tuning, the lack of data does not impede on the model's ability to learn as significantly.

In our final submission, we use ConvNeXt-Tiny, pretrained on ImageNet-22k, as the backbone of our model. We fine tune this model end-to-end on the provided dataset with aggressive data augmentation, label smoothing, and a cosine learning rate schedule. This approach achieves a score of 0.81818 on the Kaggle public leaderboard. The report below documents our experimental process and the reasoning behind the final design.

2 Dataset

The dataset consists of 1,079 training images distributed across 100 classes. The number of images per class ranges from 4 to 41, with a mean of 10.8 images per class. Most classes contain 10-11 images, but the distribution is uneven enough to matter for validation. The test set contains 1,036 unlabeled images, 10% of which Kaggle uses for the public score and 90% of which is used for final scoring.

For validation, we held out one image per class via a stratified split, randomly selecting one image from each class's training images, yielding a 979 / 100 train / val split. A naive random split would have been risky given the imbalance of images per class.

In our code, we had to perform some initial extra steps for label mapping of training images. The training folders are named "0" through "99", but PyTorch's `ImageFolder` indexes classes in alphabetical string order. This means that folder "10" would receive integer label 2 instead of 10, since it's the third alphabetically. This would scramble our predictions, so to resolve this, we subclassed `ImageFolder` to remap each sample's target from `int(folder_name)` rather than the default alphabetical index.

For preprocessing, all images are resized to 224×224 and normalized with the standard ImageNet mean $[0.485, 0.456, 0.406]$ and standard deviation $[0.229, 0.224, 0.225]$. This ensures that we feed the pretrained model inputs in the same numerical range that it was trained on.

Further, with the limited dataset provided to us, aggressive augmentation is essential to avoid model overfitting. So, the training pipeline applies four transformations in sequence. `RandomResizedCrop(224, scale=(0.7, 1.0))` takes a random crop covering between 70% and 100% of the original image area and resizes it to 224×224 , forcing the model to recognize classes from varying scales and positions.

`RandomHorizontalFlip()` mirrors the image with 50% probability, essentially doubling the dataset. Left-right orientation isn’t meaningful for any of the classes in this dataset so this does not impact class membership. `TrivialAugmentWide()` randomly selects one transformation per image (rotation, color jitter, shear, brightness, contrast, and others) at a random magnitude. Finally, `RandomErasing(p=0.25)` blanks out a random rectangular region of the image with 25% probability, preventing the model from over-relying on any single image region.

3 Implementation

3.1 Model

Our final model uses ConvNeXt-Tiny as the backbone, accessed via the `timm` library (`convnext_tiny.fb_in22k_ft_in1k`). ConvNeXt is a modern convolutional architecture introduced in 2022, designed to match the performance of vision transformers while retaining the data efficiency of traditional CNNs. The “Tiny” variant has approximately 28 million parameters, making it well-suited for a T4 GPU and small enough to avoid severe overfitting on a 1,000-image dataset.

We chose this specific checkpoint because of its pretraining: it was first trained on ImageNet-22k (14 million images across 21,841 classes) and then fine-tuned on ImageNet-1k. The 22k pretraining is significantly richer than ImageNet-1k-only pretraining, and as we describe in the Experiments section, this choice was responsible for the largest accuracy gain in the project.

For the classification head, `timm` automatically replaces the pretrained 1,000-class linear layer with a fresh 100-class layer when `create_model` is passed `num_classes=100`. No other architectural modifications (e.g., dropout, pooling, custom head) were made.

The fine-tuning strategy we used was full end-to-end fine-tuning with a small learning rate, rather than freezing the backbone or staged unfreezing. Full fine-tuning allows the model to adapt its internal feature representations to the specific 100 classes in this dataset, generally producing higher accuracy than keeping the backbone frozen. The tradeoff is that aggressive weight updates can damage the useful pretrained features, so we use a small learning rate (3×10^{-4} with AdamW) to keep updates conservative.

3.2 Training

The loss function is cross-entropy with label smoothing (smoothing parameter 0.1). Label smoothing replaces the one-hot training targets with a softer distribution that puts most of the probability mass on the correct class but spreads a small amount uniformly across all classes. This discourages the model from becoming overconfident in its predictions and improves calibration. This is particularly valuable on a small dataset, where the model could otherwise memorize training labels at 100% confidence.

The optimizer is AdamW with learning rate 3×10^{-4} and weight decay 10^{-4} . AdamW is the standard choice for fine-tuning modern image classification models. It adapts step sizes per parameter and handles weight decay correctly. The learning rate is intentionally small because large updates would push the pretrained weights away from the already useful features.

The learning rate is decayed across training using a cosine annealing schedule, which smoothly reduces the LR from 3×10^{-4} to 0 over the course of 30 epochs following the shape of a cosine curve. Early epochs use the full learning rate for fast progress; later epochs use very small learning rates for fine refinement.

Training runs for 30 epochs at batch size 32 with mixed-precision (FP16) computation enabled via PyTorch’s `torch.cuda.amp` utilities. Mixed precision uses 16-bit floating point arithmetic where it is numerically safe, providing faster training.

The hardware environment is Google Colab’s free tier, which provides an NVIDIA T4 GPU with 15 GB of VRAM. The software stack is PyTorch 2.x (Colab default), `timm` for model loading, and `torchvision` for data loading and augmentation.

4 Experiments

Throughout the development process, we tried multiple different approaches and tweaked parameters to observe their effects on validation and Kaggle accuracy.

4.1 EfficientNetV2-S

We first tried using EfficientNetV2-S pretrained on ImageNet-1k (`tf_efficientnetv2_s.in21k_ft_in1k`) as the backbone of our model. We used the full training pipeline described in Section 3 (cross-entropy with label smoothing, AdamW with cosine schedule, 30 epochs) and the augmentation stack described in Section 2. This configuration achieved 0.664 on the Kaggle public leaderboard with a peak validation accuracy of 0.72. This served as a baseline for us to confirm that the full training pipeline was implemented correctly (label remapping, data loading, augmentation, submission format, etc.) and to get a model with decent accuracy.

4.2 ConvNeXt-Tiny with ImageNet-22k Pretraining

As our first attempt at a significant improvement, we swapped the backbone from EfficientNetV2-S to ConvNeXt-Tiny pretrained on ImageNet-22k (`convnext_tiny.fb.in22k_ft_in1k`). No other changes were made. ImageNet-22k contains roughly 14 million images across 21,841 classes, compared to ImageNet-1k’s 1.28 million images across 1,000 classes. So, the richer pretraining gives the model more general-purpose features that transfer better to new, low-data tasks. Validation accuracy rose from 0.72 to 0.77, and the Kaggle score jumped from 0.664 to 0.818, a 15-point improvement from just the model swap.

4.3 MixUp / CutMix Augmentation

The training curve for ConvNeXt-Tiny showed clear overfitting, as training accuracy reached around 99%, while validation plateaued at 77%. To address this, we added batch-level augmentation: MixUp and CutMix, applied alternately with probability 1.0 via `timm`’s Mixup utility. MixUp blends two images and their labels proportionally; CutMix pastes a rectangular patch from one image onto another with labels weighted by area. Both have been shown to reduce overfitting in low-data settings.

Validation accuracy improved marginally from 0.77 to 0.78, but the Kaggle score dropped from 0.818 to 0.782. Our interpretation is that ConvNeXt-22k’s pretrained features were already generalizing well, and the additional regularization from batch-level mixing over-smoothed the model’s predictions on the test set. Given the score drop, we reverted to the configuration without mixup.

4.4 Larger and Newer ConvNeXt Variants

We then tested whether more capacity or a newer architecture variant would help, again as simple one-variable swaps of the backbone. First, we tested ConvNeXtV2-Tiny (`convnextv2_tiny.fcmae_ft_in22k_in1k`), the second-generation ConvNeXt with self-supervised pretraining via masked autoencoding. This resulted in a Kaggle score of 0.809. Next, we tested ConvNeXt-Small (`convnext_small.fb.in22k_ft_in1k`), which is from the same family as our previous model, but with 50 million parameters instead of 28 million parameters. This resulted in a Kaggle score of 0.791.

Both underperformed the ConvNeXt-Tiny V1 baseline. The most likely explanation is that with only around 10 training images per class, additional model capacity primarily enables additional memorization rather than better generalization. ConvNeXt-Tiny appears to be close to the right size for this dataset.

4.5 DINOv2 ViT-Small

We then attempted to train a self-supervised vision transformer as a second model that could be ensembled with ConvNeXt-Tiny. DINOv2 is known for producing strong transferable features and is a common choice for few-shot transfer learning, so it seemed promising on paper.

Despite this, training plateaued at 18% validation accuracy after 20 epochs, dramatically worse than ConvNeXt’s 77%. To diagnose, we froze the backbone entirely and trained only the classifier head for 5

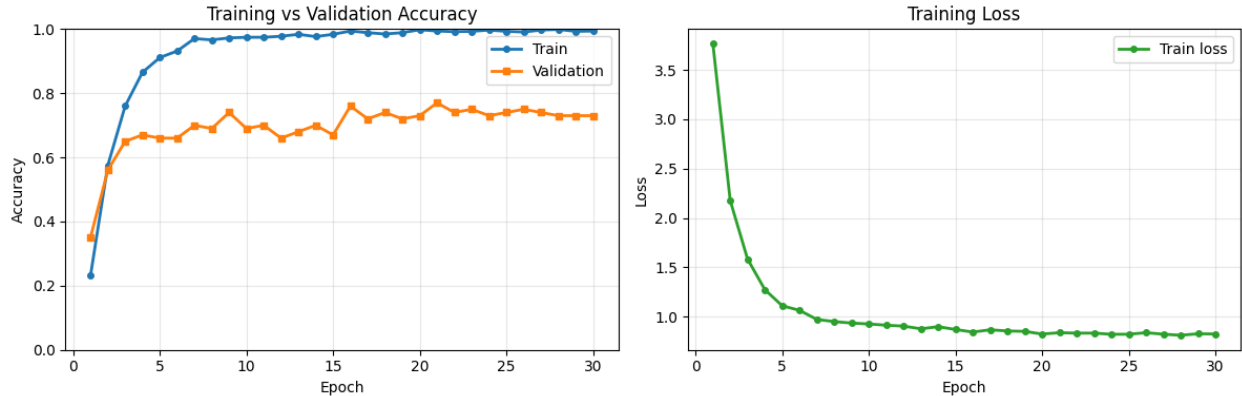


Figure 1: Training and validation accuracy (left) and training loss (right) for the final ConvNeXt-Tiny model over 30 epochs. Training accuracy reaches 99% by epoch 10, while validation accuracy plateaus at 77% from epoch 9 onward.

epochs. Validation accuracy stalled at 16%, confirming that the issue was not fine-tuning dynamics but the features themselves. DINOv2’s pretrained representations do not transfer well to this particular dataset.

We did not pursue DINOv2 further after this. The most likely explanation is a domain mismatch between DINOv2’s LVD-142M pretraining distribution and the specific class distribution sampled for this project.

4.6 Test-Time Augmentation

Finally, we applied horizontal-flip test-time augmentation to the best ConvNeXt-Tiny checkpoint. At inference, each test image is passed through the model both in its original orientation and flipped, and the softmax outputs are averaged before taking the argmax.

For this submission, the public Kaggle score did not change (0.818 with or without TTA). One plausible explanation is that the model’s errors on this dataset are largely systematic rather than orientation-sensitive, so averaging two views does not flip predictions for borderline cases. Another is that the public leaderboard sample is small enough that a real 1% improvement would only flip a single prediction and might be lost to rounding.

5 Results

The final submission achieves a score of 0.818 on the Kaggle public leaderboard, with a best validation accuracy of 0.77 reached at epoch 21. This final submission was the approach described in the ConvNeXt-Tiny with ImageNet-22k Pretraining subsection of the previous section. Figure 1 shows the training and validation accuracy curves alongside the training loss.

The curve shows two key dynamics. First, the pretrained features are immediately useful: validation accuracy jumps from 0.35 at epoch 1 to 0.65 by epoch 3, demonstrating that most of the learning comes from adapting the model’s classification head rather than reshaping the backbone. Second, training and validation diverge sharply. By epoch 10, training accuracy reaches 0.97 while validation plateaus at 0.70 and never substantially improves. This 22-point gap is overfitting in its classic form. The model has the capacity to memorize the 979 training examples, but the limited dataset offers no further generalization signal beyond what the pretrained features already provide.

6 Discussion

Through the experiments described above, it was apparent in training a model for this classification problem that the choice of pretrained backbone dominated all other design decisions. The single change from

ImageNet-1k to ImageNet-22k pretraining produced a 15-point improvement on Kaggle. In a low-data regime where the model sees only around 10 examples per class, the quality of the starting features matters more than the architecture’s recency, the model’s size, or the augmentation strategy.

The negative results were just as informative. MixUp and CutMix hurt performance, suggesting that ConvNeXt-22k’s features were already well-regularized for this task and adding batch-level mixing over-smoothed the model. Larger ConvNeXt variants also underperformed, indicating that more parameters meant more capacity to memorize the small training set without correspondingly more generalization. DINOv2 failed to transfer, with a diagnostic experiment confirming the issue lay in the pretrained features themselves rather than fine-tuning dynamics. Each of these results pointed in the same direction: this dataset is small enough that techniques designed for larger-scale problems can actively hurt rather than help.

The clearest limitation of the final model is the train/val gap visible in Figure 1. Although the model generalizes well in absolute terms, it has clearly run out of useful signal from the training set well before training ended. Several next steps could plausibly push the score further. First, training the same ConvNeXt-Tiny configuration with several different random seeds and averaging their predictions could produce an improvement. We did not have time to run multiple seeds, but this form of ensembling could be a fairly simple way to extract more accuracy from the current model. Next, DINOv2 failed, but other self-supervised models (BEiT, MAE, SigLIP) have different pretraining objectives and might transfer better. Lastly, the current validation set has only 100 images, so its accuracy estimate is noisy. A 5-fold CV setup would give a tighter performance estimate at the cost of $5\times$ training time, and the 5 trained models could be ensembled for inference.

7 Reproducibility

To ensure reproducibility of our results, we fix the seed at 42 across Python’s `random` module, NumPy, and PyTorch (both CPU and CUDA generators). This makes the train/validation split, weight initialization, augmentation samples, and batch ordering all reproducible across runs on the same hardware.

As a note on determinism, bit-level determinism flags (`torch.use_deterministic_algorithms`, `cuda.deterministic`) were not enabled, as they slow training noticeably and the rubric for this project calls for “similar means and standard deviations” rather than identical runs. Results should reproduce within approximately $\pm 1\%$ on the same hardware.

To reproduce our results in the same environment, use the Google Colab free tier with an NVIDIA T4 GPU and PyTorch 2.x (Colab default), and `timm` installed via `pip install -q timm`.

To run the code, place the unzipped dataset in Google Drive at `MyDrive/ucsc-cse-144-spring-2026-final-project/` (with subdirectories `train/`, `test/`, and the file `sample_submission.csv`). Then, open `CSE144FinalProject.ipynb` in Google Colab with a T4 GPU runtime. Run all cells in order. Training should take under 30 minutes. The notebook saves the best checkpoint to Google Drive and writes `submission.csv` to both Google Drive and the local Colab filesystem.

8 Team Contributions

This was a solo submission. All work (code, experiments, presentation, and report) was completed by Ayush Manocha.

9 References

1. Liu, Z., Mao, H., Wu, C., Feichtenhofer, C., Darrell, T., & Xie, S. (2022). *A ConvNet for the 2020s*. CVPR.
2. Wightman, R. (2019). *PyTorch Image Models (timm)*. <https://github.com/huggingface/pytorch-image-models>
3. Müller, S. G., & Hutter, F. (2021). *TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation*. ICCV.

4. Zhang, H., Cissé, M., Dauphin, Y. N., & Lopez-Paz, D. (2018). *mixup: Beyond Empirical Risk Minimization*. ICLR.
5. Yun, S., Han, D., Oh, S. J., Chun, S., Choe, J., & Yoo, Y. (2019). *CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features*. ICCV.
6. Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). *Rethinking the Inception Architecture for Computer Vision*. CVPR. (label smoothing)
7. Oquab, M., et al. (2024). *DINOv2: Learning Robust Visual Features without Supervision*. Transactions on Machine Learning Research.
8. TorchVision documentation. <https://pytorch.org/vision/stable/models.html>